

PYTHON FUNDAMENTALS

CE IT 1015 | Session 1: Basics & Git Setup

Instructor: Lance Muwayi

Structure For Session 1

Week 1

STRUCTURE OF THE DAY



Part 1

Python Introduction
& Core Basics.
Understanding types
and variables.



Part 2

Workshop: Your first
Python programs
and logic exercises.



Part 3

Git Fundamentals.
Version control and
GitHub
configuration.



Part 4

Workshop: Git setup
and your first
repository commit.

PART 1: Python Basics

LEARNING OBJECTIVES

- ✓ **First Steps:** Write your first Python program today.
- ✓ **Data Mastery:** Understand variables and standard data types.
- ✓ **Operations:** Use arithmetic and comparison operators.
- ✓ **Interaction:** Master the `print()` function and code comments.

WHAT IS PYTHON?

A Universal Language

Created by **Guido van Rossum** in 1991, Python is a high-level, interpreted programming language known for its readability.

- ✓ **Versatile:** Web, Data Science, AI.
- ✓ **Popular:** Top language in 2024.
- ✓ **Elegant:** Easy to learn, powerful to use.



WHY LEARN PYTHON?



Python offers a strong community, powerful libraries, and wide industrial adoption.

YOUR FIRST PROGRAM

hello.py

```
print('Hello, World!')
```

A single line of code is all it takes to start your journey.

How to Run

- 1 Save as hello.py
- 2 Open Terminal/Command Prompt
- 3 Type: python hello.py

BASIC DATA TYPES

Integer (int)

Whole numbers like
42 or -10.

Float

Decimal numbers
like 3.14 or -0.5.

String (str)

Text data like 'Hello'
or "World".

Boolean (bool)

Logical values: True
or False.

VARIABLES: STORING DATA

Variable Name	Value	Inferred Type
name	'Alice'	String (str)
age	25	Integer (int)
height	5.6	Float
is_student	True	Boolean (bool)

Python uses **dynamic typing**: it determines the type automatically!

VARIABLE NAMING RULES

Do This

-  Start with a letter or underscore.
-  Use letters, numbers, underscores.
-  Case-sensitive: age ≠ Age.

Avoid This

-  Starting with a number: 1st_place.
-  Reserved keywords: if, for.
-  Spaces in names: my var.

ARITHMETIC OPERATORS

Basic Operations

$10 + 5 \rightarrow$ Addition (15)

$10 - 5 \rightarrow$ Subtraction (5)

$10 * 5 \rightarrow$ Multiplication (50)

$10 / 5 \rightarrow$ Division (2.0)

Advanced Operations

$10 // 3 \rightarrow$ Integer Division (3)

$10 \% 3 \rightarrow$ Modulus/Remainder (1)

$2 ** 3 \rightarrow$ Exponentiation (8)

COMPARISON OPERATORS

$5 > 3 \rightarrow \text{True}$ (Greater than)

$10 < 5 \rightarrow \text{False}$ (Less than)

$7 == 7 \rightarrow \text{True}$ (Equal to)

$4 != 4 \rightarrow \text{False}$ (Not equal)

$6 >= 6 \rightarrow \text{True}$ (Greater or equal)

$8 <= 7 \rightarrow \text{False}$ (Less or equal)

Comparisons always return a **Boolean** (True or False).

LOGICAL OPERATORS

AND

Returns True if **both** conditions are True.

```
age >= 16 and has_license
```

OR

Returns True if **at least one** is True.

```
age < 18 or age > 65
```

THE PRINT() FUNCTION

Modern F-Strings

The best way to format text in Python 3.6+.

```
age = 25  
print(f'I am {age} years old')
```

Multiple items can be separated by commas in a standard print statement: `print('Hello', name)`.



CODE DOCUMENTATION

Single Line

Use # for brief notes.

```
# This is a comment  
x = 10 # Inline comment
```

""" Multi-Line """

Use triple quotes for long explanations.

```
'''  
This is for long  
block comments.  
'''
```

PART 2: Workshop

First Python Programs

EX 1: VARIABLES & PRINT

Task

Create about_me.py with variables for name, age, height, and a boolean for "likes programming".

Formatting

Print all info using **f-strings** for a professional look.

Notes

Add comments explaining what each variable stores.

EX 2: CALCULATOR

Variables

```
num1 = 15  
num2 = 4
```

Operations

Print results for: +, -, *, /, //,
and %.

Clarity

Use f-strings to label each
result clearly.

EX 3: AGE CHECKER

Logic

Create `your_age` and
`voting_age = 18`.

Check

Use `>=`, `<`, and `==`
comparisons.

Output

Print descriptive messages for
each logical check.



10 MIN BREAK

Stretch, hydrate, and prepare for Git setup.

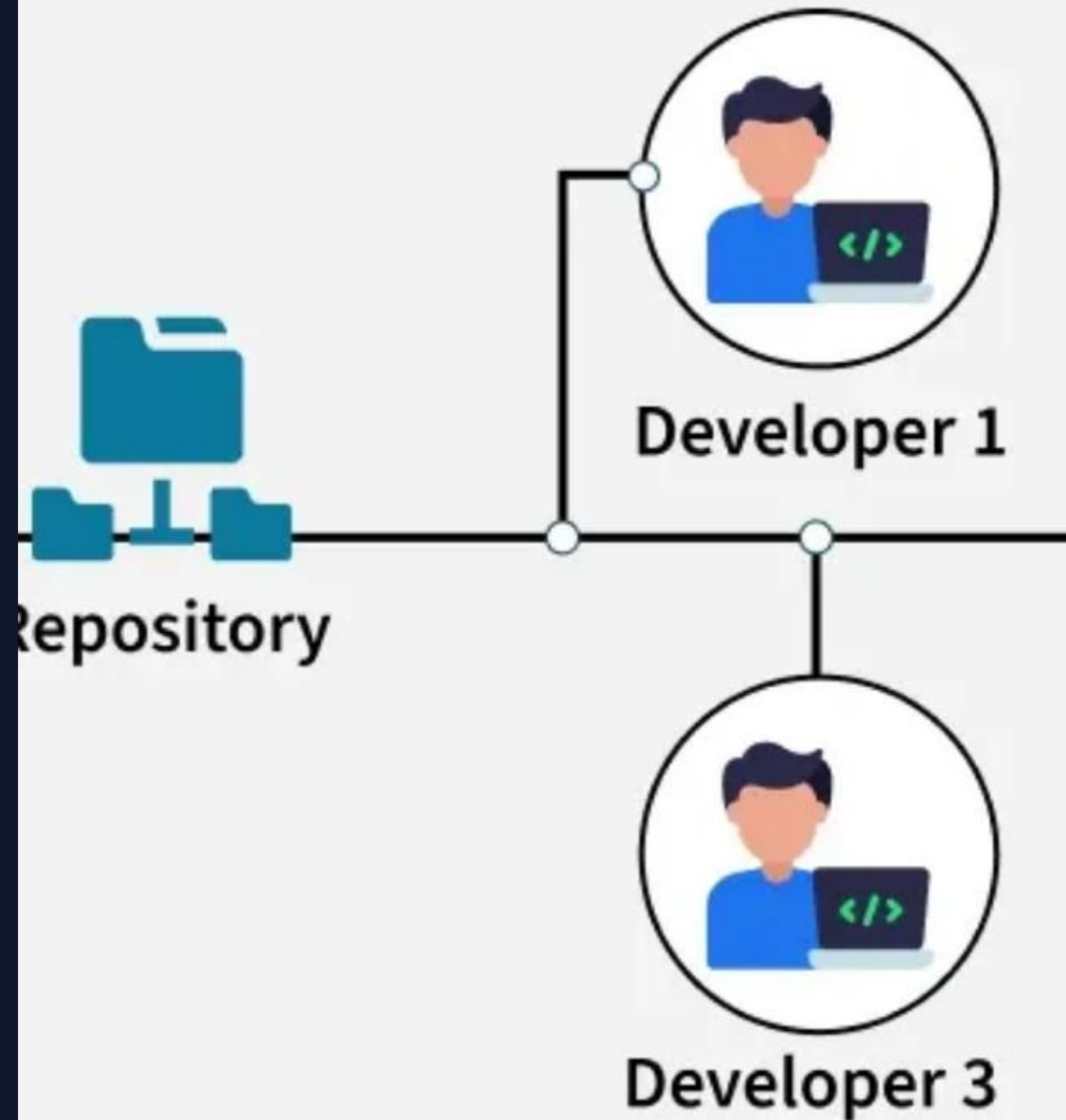
PART 3: Git & Version Control

THE VALUE OF VC

Why use Version Control?

A system for tracking changes to files over time. It's like "Track Changes" in Word but for code.

Essential for collaboration, tracking history, and experimenting safely without fear of breaking the project.



WHAT IS GIT?



Industry Standard

Created by Linus Torvalds in 2005. It is a distributed system, meaning everyone has the full history locally.

- ✓ Free and Open Source.
- ✓ Works on Windows, Mac, Linux.
- ✓ Used by millions worldwide.

GIT VS GITHUB

Git (The Engine)

Software that runs locally on your computer. It tracks the actual changes in your files.

GitHub (The Platform)

A website for hosting repositories. It's cloud storage for code and a collaboration hub.

INSTALLING GIT



Download: git-scm.com/downloads



Windows: Git for Windows (includes Git Bash).



Mac: Use Homebrew or Xcode Tools.



Verify: Run `git --version` in terminal.

CONFIGURING GIT

Identity Setup

```
git config --global user.name 'Name'  
git config --global user.email 'email'
```

This identifies you in the history. Do this **once**.

Check Settings

```
git config --list
```

View your current configuration settings anytime.

GITHUB ACCOUNT SETUP

 Go to github.com and choose a professional username.

 Use a personal email and verify your address.

 **Security:** Enable 2FA immediately.

 Add a profile picture to your showcase.

GIT BASICS WORKFLOW

ADD

Stage Files

PUSH

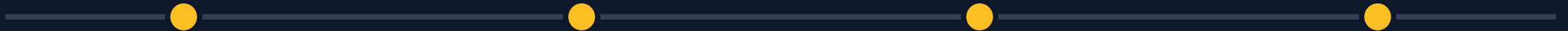
Upload to Cloud

INIT

Initialize Repo

COMMIT

Save Snapshot



YOUR FIRST REPOSITORY

Setup

```
cd project-folder  
git init
```

Creates the hidden .git folder.

Check State

```
git status
```

See which files are "Untracked".

MAKING A COMMIT

1. Stage

```
git add hello.py
```

Or use `git add .` for all files.

2. Commit

```
git commit -m 'initial message'
```

Creates a checkpoint in your project history.

COMMIT MESSAGE TIPS

Good

- ✓ 'Add login functionality'
- ✓ 'Fix bug in calculator'
- ✓ 'Update README'

Bad

- ✗ 'stuff'
- ✗ 'changes'
- ✗ 'idk fixed it'

CONNECTING TO GITHUB

Add Remote

```
git remote add origin https://github.com/...
```

Connects your local repo to your GitHub host.

Push Files

```
git push -u origin main
```

Uploads your local history to the cloud server.

PART 4: Workshop

Git Setup & First Commit

EX 4: GIT CONFIGURATION

Terminal

Open command prompt. Set name and email using config commands.

GitHub

Sign up at github.com. Verify email and enable 2FA.

Verify

Run `git config --list` to confirm your credentials.

EX 5: FIRST COMMIT

Folder

Create python-week1. Move about_me.py inside.

Workflow

git init → git add → git commit.

Log

Run git log to see your successful commit history.

SESSION 1 SUMMARY

- ✓ Python Basics: Types, variables, and operators.
- ✓ Functionality: Output with `print()` and f-strings.
- ✓ Git: Initializing repositories and staging/committing files.
- ✓ Remote: Connecting to GitHub for portfolio hosting.

HOMework ASSIGNMENT



Complete the Session 1 Quiz.



Create 3 new Python programs using varied operators and variables.



Push your new Git repository to GitHub.



Read about Strings and Methods for Session 2.

QUESTIONS?

Session 2: Variables, Strings & Git Workflow

See you next time! 🚀

IMAGE SOURCES



<https://upload.wikimedia.org/wikipedia/commons/thumb/c/c3/Python-logo-notext.svg/1280px-Python-logo-notext.svg.png>

Source: en.wikipedia.org



<https://media.geeksforgeeks.org/wp-content/uploads/20251106175516094713/server.webp>

Source: www.geeksforgeeks.org